

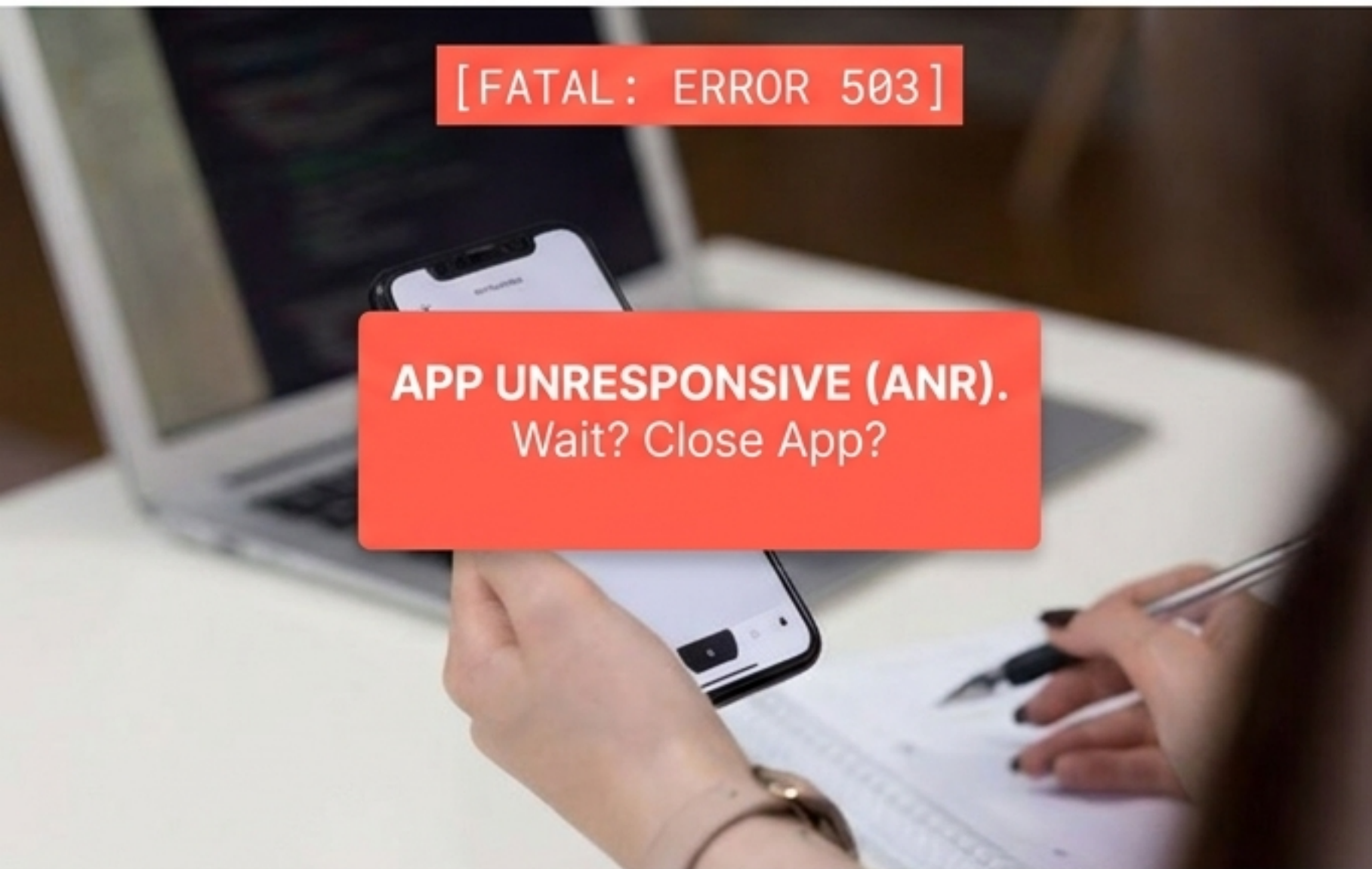
PRUEBAS EN EL PROCESO DE DESARROLLO SOFTWARE

Garantizando la calidad, el rendimiento y la viabilidad técnica desde el código hasta el usuario final.



El coste de la baja calidad

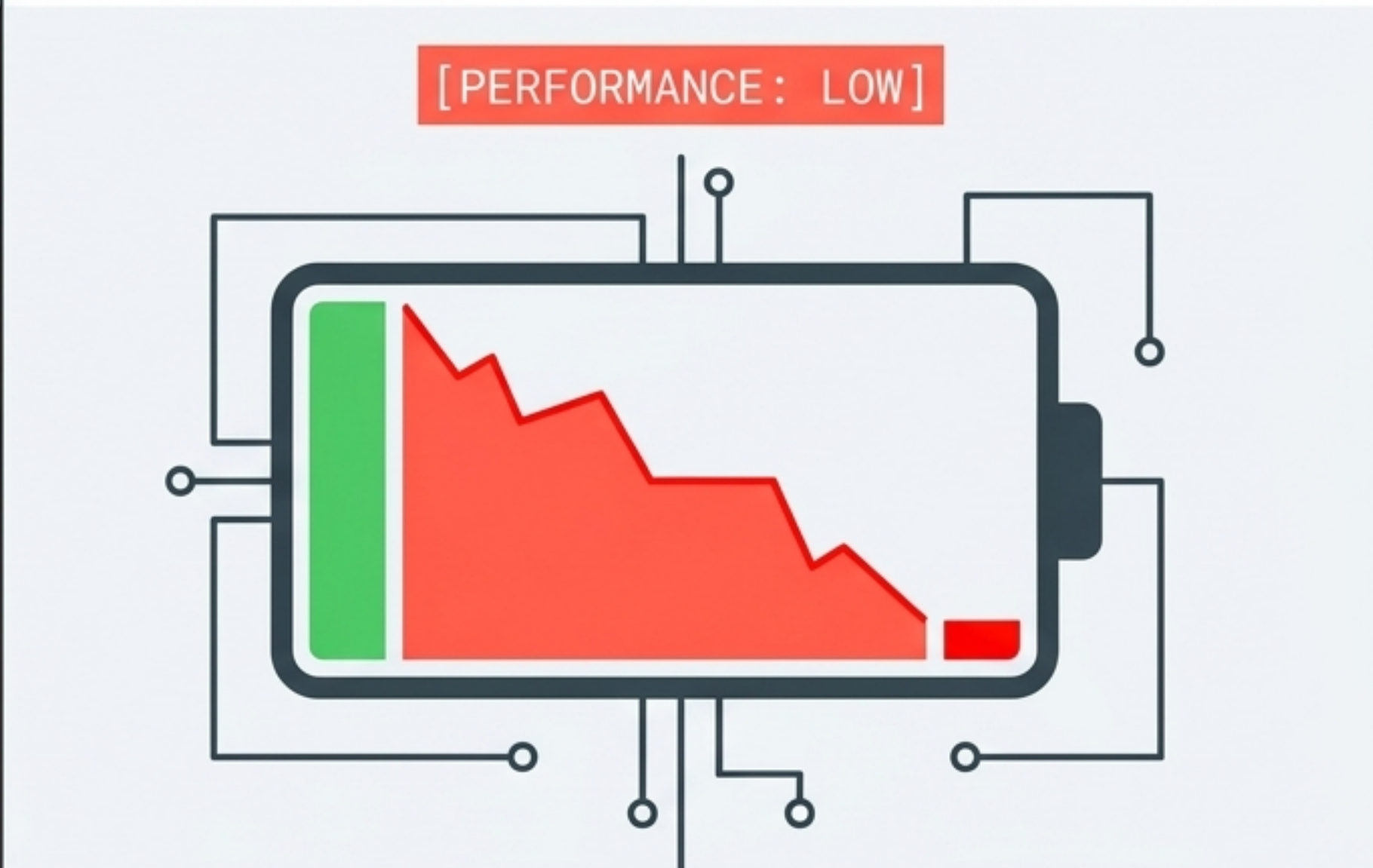
Destacar en el mercado móvil actual exige pruebas rigurosas. La calidad es el filtro definitivo.



[FATAL: ERROR 503]

APP UNRESPONSIVE (ANR).
Wait? Close App?

Bloqueos (ANR). Las aplicaciones que no responden son desinstaladas inmediatamente. Google y Apple penalizan y retiran estas apps de sus tiendas.



[PERFORMANCE: LOW]

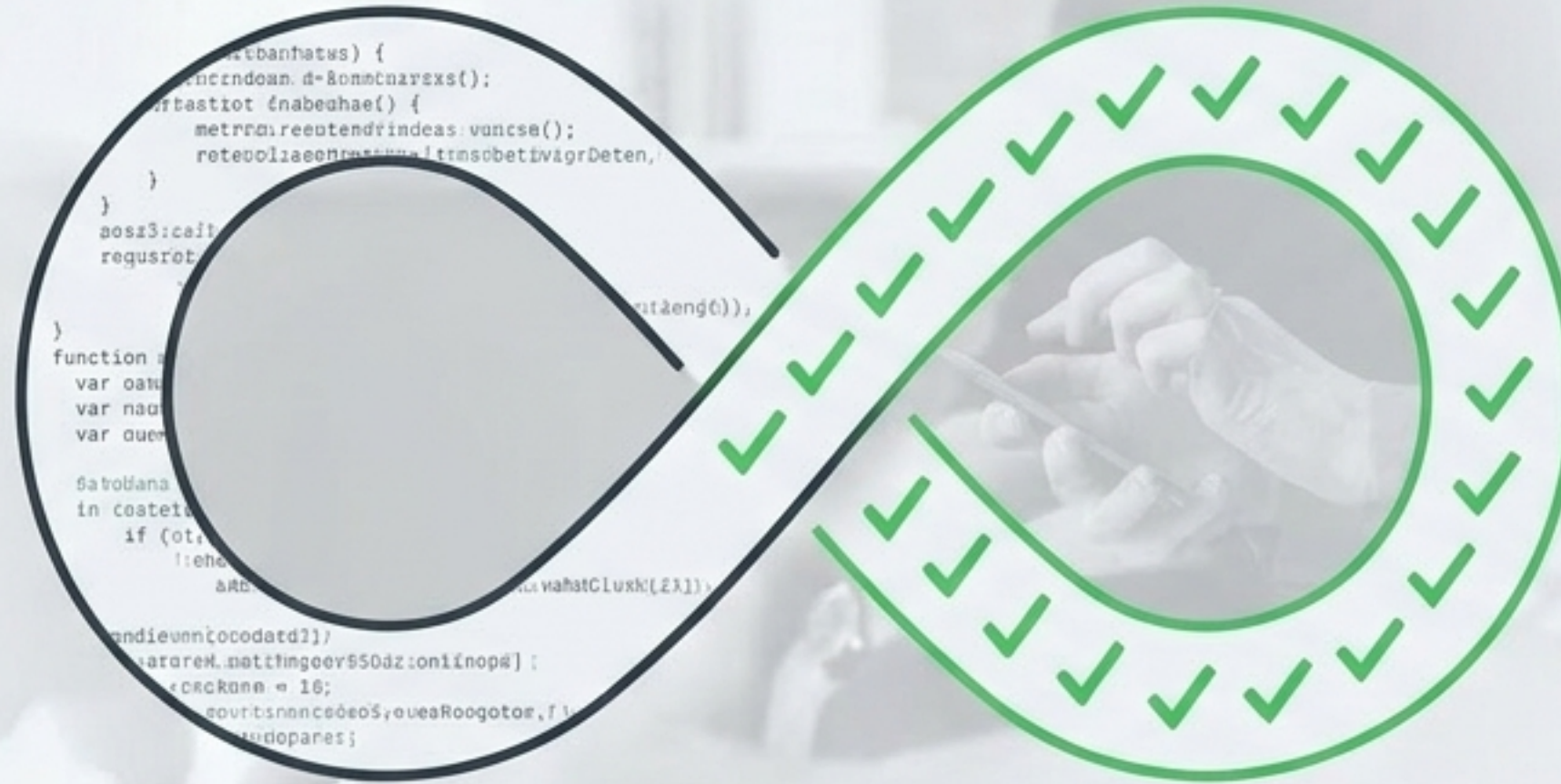
Consumo de Batería. El mal uso de los recursos físicos del dispositivo ahuyenta a los usuarios.

Las pruebas como motor del desarrollo

Las pruebas requieren una inversión de tiempo crítica (ejecución + comprobación).
No son un paso final, sino la base de metodologías modernas.

Scrum

Basa su filosofía en un conjunto robusto de pruebas para mantener velocidad de desarrollo y calidad iterativa.



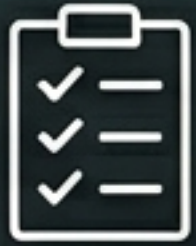
TDD (Test-Driven Development)

Las pruebas guían el desarrollo. Se escriben antes que el código para detectar errores de forma inmediata y evitar regresiones.

Clasificación de Pruebas: Ejecución

 Pruebas Estáticas		 Pruebas Dinámicas	
Definición	Análisis lógico previo sin ejecutar el código de la aplicación.	Definición	Requieren la ejecución real del código.
Objetivo	Revisión de documentos, requisitos y estructura lógica.	Técnicas	Caja negra, caja blanca.
Resultado	Detecta fallos lógicos antes de la compilación.	Resultado	Comprueba el comportamiento real frente al comportamiento esperado.

Clasificación de Pruebas: Objetivos



Pruebas Funcionales: ¿Hace lo que debe hacer?

Comprueba que la implementación coincide con los requisitos capturados con el cliente.

Resolución - Caso Práctico 1: Para validar que todas las funcionalidades solicitadas por el cliente han sido desarrolladas, la estrategia exacta es ejecutar pruebas funcionales masivas y actualizar la planificación si faltan requisitos.



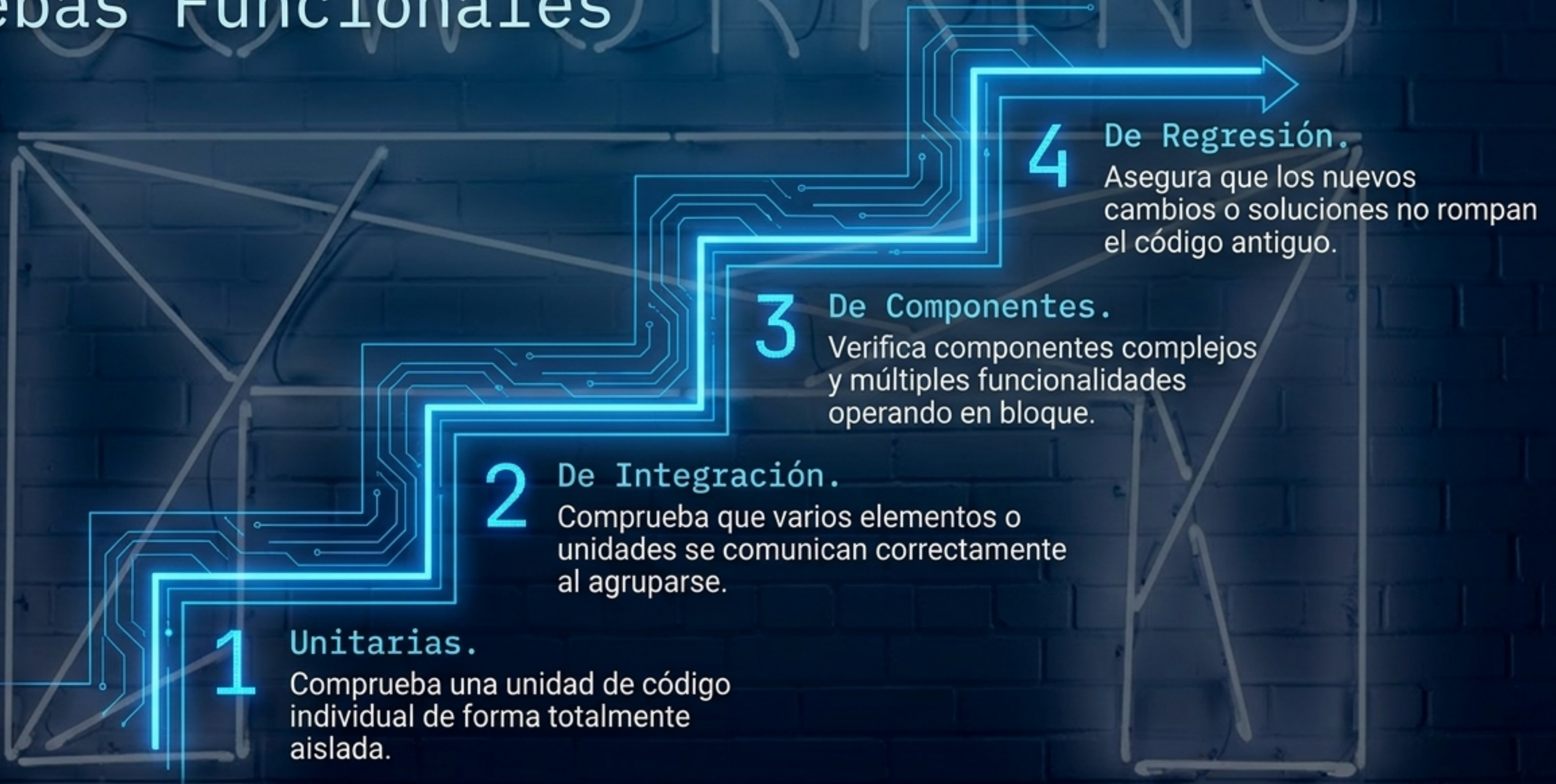
Pruebas No Funcionales: ¿Cómo de bien lo hace?

Se centra en el rendimiento, la seguridad, la usabilidad y el consumo de batería.

Anatomía de un Caso de Prueba



El alcance de las Pruebas Funcionales



El mito de las Pruebas Unitarias

El Planteamiento: El equipo asume que, si pasan todas las pruebas unitarias, el software está 100% libre de errores.



[PASS]

El Problema: Las pruebas unitarias ignoran el entorno y las dependencias.

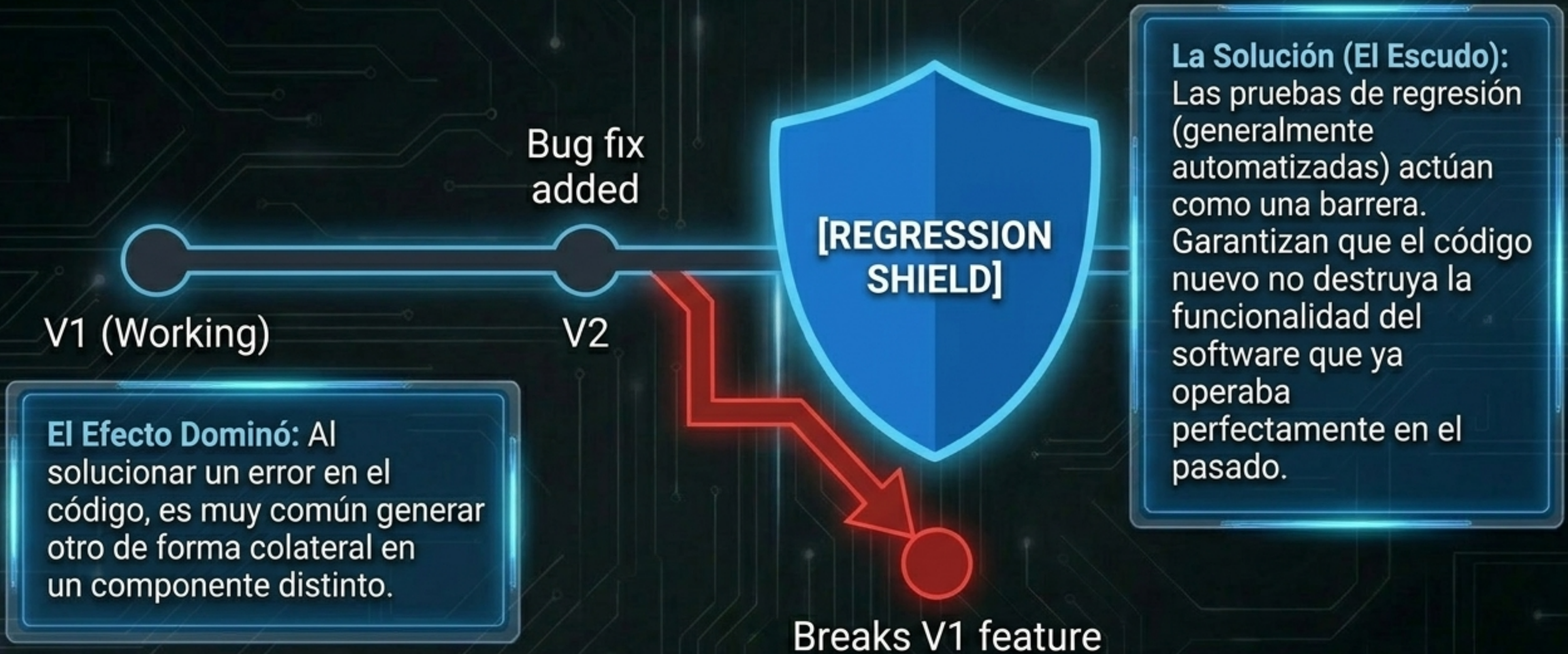


[FAIL]

No saben cómo interactúa un componente con el resto del sistema.

La Solución: Es absolutamente obligatorio complementar con Pruebas de Integración para simular el comportamiento real en conjunto.

Protegiendo el pasado: Pruebas de Regresión



La Regla de Oro del Testing



“Las pruebas nunca pueden asegurar que el software esté libre de errores.”

La Realidad

Los entornos, dispositivos y usuarios escapan a nuestro control. Lo único que una prueba puede garantizar es que el error no se reprodujo bajo las condiciones específicas probadas.

El Enfoque Técnico

Por eso, cuando un cliente reporta un fallo, la primera pregunta de un desarrollador siempre debe ser: “¿Bajo qué contexto y circunstancias exactas ocurrió?”

Planificación del tiempo: Gantt vs. PERT

Contexto: El desarrollo real sufre imprevistos (bajas médicas, retrasos en despliegues). La planificación de pruebas requiere prever tiempos de margen, conocidos como holguras.



Diagrama de Gantt: Inadecuado para esta fase. Su estructura rígida no permite indicar fácilmente las holguras frente a imprevistos.



Método PERT: La elección correcta.

Método PERT: La elección correcta. Permite introducir dependencias complejas y calcular tiempos de contingencia, asegurando que el equipo de QA tenga el tiempo real que necesita.

Conclusiones Clave



Calidad = Supervivencia en el mercado. Las pruebas evitan bloqueos críticos (ANR) y mantienen la aplicación viable para los usuarios.



Planificación desde el inicio. Las pruebas consumen tiempo de desarrollo, pero son el pilar de metodologías ágiles como Scrum y TDD.



Diversidad de enfoques. Se requieren pruebas estáticas, dinámicas, funcionales y no funcionales para atrapar distintos tipos de fallos.



Dependencias: Integración y Regresión. Una prueba unitaria aislada no garantiza un sistema funcional; el código debe probarse en conjunto y protegerse contra el efecto dominó.